

12

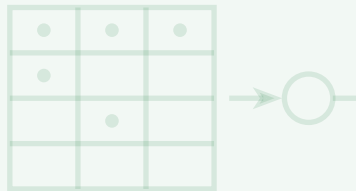
Лекция

24–25 ноября

Булевы функции и SAT

Константин Чухарев

Дискретная математика — Осень 2026



Булевы функции

Логика — алгебра высказываний.

— Джордж Буль

От логики к железу

Джордж Буль в 1854 году алгебраизировал логику: конъюнкция стала умножением, дизъюнкция — сложением, отрицание — дополнением.

В 1937 году Клод Шеннон показал, что та же алгебра описывает поведение реле: замкнутая цепь — истина, разомкнутая — ложь.

Логика XIX века стала железом XX века: каждая булева формула — цепь переключателей.

Булева функция

Определение 1

Булева функция: $f : \{0, 1\}^n \rightarrow \{0, 1\}$.

Аргументы и значение — биты.

Функция полностью задаётся таблицей истинности из 2^n строк.

Знакомые связки — конъюнкция, дизъюнкция, импликация — частные случаи: каждая берёт пару бит и выдаёт бит.

«Задать функцию» и «заполнить таблицу» — одно и то же: никакого другого описания произвольной булевой функции нет.

Таблица истинности

Пример

$f(x, y) = x \wedge y$ истинна ровно в одной строке и гаснет от любого нуля:

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

Таблица — полное, но громоздкое описание: строк 2^n .

Формула и схема — компактные описания для функций, у которых структура есть.

Сколько существует булевых функций?

Теорема 1 (Подсчёт)

Различных булевых функций от n переменных ровно 2^{2^n} .

Доказательство

Докажем подсчётом таблиц истинности.

В таблице 2^n строк, и выход в каждой строке выбирается независимо: 0 или 1.

Значит, таблиц 2^{2^n} , и две разные таблицы задают разные функции.

Две шкалы роста

строк таблицы: 2^n , булевых функций: 2^{2^n} .

Пример

- $n = 2$: $2^4 = 16$ — все классические вентили.
- $n = 5$: $2^{32} > 4$ миллиарда.
- $n = 10$: 2^{1024} — больше, чем атомов в наблюдаемой вселенной.

Рост 2^{2^n} быстрее любой экспоненты: таблица — полное, но безнадежно громоздкое описание функции.

Бинарных функций шестнадцать

При $n = 2$ у каждой из шестнадцати функций есть имя. Остальные восемь — импликация $x \rightarrow y$, отрицание обратной, проекции, их отрицания и константы 0, 1.

Имя	Формула	Смысл
Конъюнкция	$x \wedge y$	AND
Дизъюнкция	$x \vee y$	OR
Отрицание импликации	$\neg(x \rightarrow y)$	антиимпликация
Обратная импликация	$y \rightarrow x$	обратная
XOR	$x \oplus y$	входы различны
Эквивалентность	$x \equiv y$	входы совпадают
Штрих Шеффера	$x \uparrow y$	NAND
Стрелка Пирса	$x \downarrow y$	NOR

Нормальные формы

Формула — проект схемы, а нормальная форма — стандартизованная укладка вентиляей и проводов.

Ниже — рабочий словарь сборки: минтерм, макстерм и два рецепта, по которым форма строится из таблицы.

- ДНФ — дизъюнкция конъюнкций литералов, КНФ — конъюнкция дизъюнкций. Каждая конъюнкция литералов — **терм**, каждая дизъюнкция литералов — **дизъюнкт**.
- Совершенные формы СДНФ и СКНФ единственны — канонический портрет функции.

Минтерм и макстерм

Определение 2 (Минтерм)

Минтерм — конъюнкция литералов, в которую каждая переменная входит ровно один раз.

Определение 3 (Макстерм)

Макстерм — дизъюнкция литералов, в которую каждая переменная входит ровно один раз.

Конъюнкция истинна лишь когда истинны все её литералы: минтерм равен 1 ровно на одной строке таблицы и 0 на всех остальных.

Дизъюнкция ложна лишь когда ложны все её литералы: макстерм равен 0 ровно на одной строке и 1 на всех остальных.

Дуальная пара: минтерм — детектор единичной строки, макстерм — детектор нулевой.

Детекторы строк

Пример (Детекторы при $n = 3$)

Минтерм $x \wedge \bar{y} \wedge z$ истинен только на наборе $(1, 0, 1)$: чтобы конъюнкция засветилась, все три литерала должны стать единицами разом.

Макстерм $x \vee \bar{y} \vee z$ ложен только на наборе $(0, 1, 0)$: чтобы дизъюнкция погасла, все три литерала должны погаснуть разом.

Минтермов и макстермов ровно 2^n — по одному на каждую строку таблицы.

Каждый минтерм и каждый макстерм однозначно соответствует строке таблицы истинности — на этой нумерации держится сборка совершенных форм.

Нормальные формы XOR

Пример

$x \oplus y$ истинна на наборах $(0, 1)$ и $(1, 0)$, ложна на $(0, 0)$ и $(1, 1)$.

СДНФ — по единицам:

$$(\bar{x} \wedge y) \vee (x \wedge \bar{y}).$$

СКНФ — по нулям:

$$(x \vee y) \wedge (\bar{x} \vee \bar{y}).$$

Рецепт: СДНФ собирается из минтермов единичных строк, СКНФ — из макстермов нулевых.

У $x \oplus y$ обе формы имеют по два терма: функция симметрична, экономить не на чем.

Совершенная форма избыточна

Пример

Импликация $x \rightarrow y$ истинна на трёх наборах из четырёх:

$$x \rightarrow y = (\bar{x} \wedge \bar{y}) \vee (\bar{x} \wedge y) \vee (x \wedge y).$$

Алгебраическое упрощение сжимает совершенную форму до двух литералов вместо шести:

$$x \rightarrow y = \bar{x} \vee y.$$

СДНФ единственна, но избыточна: каждый лишний литерал — лишний вентиль.

Найти самую короткую равносильную ДНФ — задача минимизации.



Задача SAT

Нет царского пути в геометрию.

— Евклид

Задача SAT

Определение 4 (Выполнимость)

Формула в КНФ **выполнима** (*satisfiable*), если существует набор значений переменных, при котором она истинна.

Задача SAT: по данной КНФ узнать, выполнима ли она.

Пример

$$\varphi = (x \vee \bar{y}) \wedge (\bar{x} \vee y \vee z) \wedge \bar{z}.$$

Набор $x = 1, y = 1, z = 0$ выполняет их разом, каждый дизъюнкт через свой литерал.

Модель — сертификат выполнимости: предъявленный набор проверяется за линейное время.

Эта простая половина потом станет первой частью теоремы Кука — Левина.

Наборы и модели

Определение 5 (Модель)

Набор значений переменных, при котором формула истинна, называется **моделью** (*model*).

Запись $\sigma \models \varphi$ читается «набор σ выполняет формулу φ ».

Каждая из n переменных независимо выбирает 0 или 1, поэтому наборов ровно 2^n .

x	y	$x \vee y$	$\bar{x} \vee y$	ψ
0	0	0	1	0
0	1	1	1	1
1	0	1	0	0
1	1	1	1	1

Столбец $\psi = (x \vee y) \wedge (\bar{x} \vee y)$ повторяет столбец y : модели $(0, 1)$ и $(1, 1)$.

Сколько наборов запрещает дизъюнкт?

Пример

Для $\varphi = (x \vee \bar{y}) \wedge (\bar{x} \vee y \vee z) \wedge \bar{z}$ дизъюнкт $\bar{z}$ ложен при $z = 1$ и запрещает $2^{3-1} = 4$ набора.

Дизъюнкт $x \vee \bar{y}$ ложен при $x = 0, y = 1$ — запрещены $(0, 1, 0)$ и $(0, 1, 1)$.

Дизъюнкт $\bar{x} \vee y \vee z$ ложен только на наборе $(1, 0, 0)$.

Перекрытие одно: набор $(0, 1, 1)$ запрещён и первым, и третьим дизъюнктом, поэтому запрещены $4 + 2 + 1 - 1 = 6$ наборов из 8, и остаются две модели: $(0, 0, 0)$ и $(1, 1, 0)$.

Дизъюнкт из k литералов запрещает ровно 2^{n-k} наборов: он ложен лишь при одном значении своих k переменных, а остальные $n - k$ свободны.

При $n = 10$ один трёхлитеральный дизъюнкт убивает $2^7 = 128$ наборов из 1024.

Выполнимость и общезначимость

Выполнимость — не общезначимость: x выполнима при $x = 1$, но тавтологией не является.

Связь точная: φ выполнима тогда и только тогда, когда $\neg\varphi$ — не тавтология.

Пример

$x \wedge \bar{x}$ — простейшая невыполнимая формула: противоречие при любом наборе.

Для SAT естественна КНФ: решатели работают именно с конъюнкцией дизъюнктов.

Любую формулу можно привести к КНФ, а схема — это уже почти КНФ.

КНФ: литералы и дизъюнкты

Определение 6 (Литерал)

Литерал — переменная x или её отрицание $\bar{x}$.

Определение 7 (Дизъюнкт)

Дизъюнкт — дизъюнкция литералов: $C = (l_1 \vee \dots \vee l_k)$.

От минтерма и макстерма дизъюнкт отличается свободой: переменная может входить сколько угодно раз или не входить вовсе.

Пример

$\bar{x} \vee \bar{y} \vee z$ — дизъюнкт из трёх литералов, ложный ровно при $x = y = 1, z = 0$.

Это детектор одной нулевой строки, которому разрешили забыть часть переменных.

КНФ

Определение 8 (КНФ)

Конъюнктивная нормальная форма (КНФ) — конъюнкция дизъюнктов:

$$\varphi = C_1 \wedge \dots \wedge C_m.$$

Пример

$\varphi = (x \vee \bar{y}) \wedge (\bar{x} \vee y \vee z) \wedge \bar{z}$ — КНФ из трёх дизъюнктов длины 2, 3 и 1.

Однолитеральный дизъюнкт — **единичный** (*unit*): $\bar{z}$ принуждает $z = 0$ в любой модели.

Единичный дизъюнкт — готовый ход решателя: по z у формулы больше нет свободы.

Приведение к КНФ

Произвольная формула не обязана быть ни в одной из форм, поэтому нужен механический маршрут.

Определение 9 (ДНФ)

Дизъюнктивная нормальная форма (ДНФ) — дизъюнкция конъюнкций литералов.

КНФ и ДНФ — два стандартных каркаса, и любой формуле их придают три шага.

- Убрать $\rightarrow$ и $\equiv$ по определениям.
- Протянуть $\neg$ внутрь по законам де Моргана, до литералов.
- Дистрибутировать $\vee$ по $\wedge$: $P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R)$.

Первые два шага сохраняют равносильность даром, и только дистрибуция платит размером.

Приведение: работа шагов

Пример

$$\varphi = (x \rightarrow y) \rightarrow z.$$

Шаги 1–2: $(\bar{x} \vee y) \rightarrow z$ превращается в $\overline{\bar{x} \vee y} \vee z = (x \wedge \bar{y}) \vee z$.

Шаг 3: дистрибуция даёт $(x \vee z) \wedge (\bar{y} \vee z)$ — КНФ из двух дизъюнктов.

Для SAT естественна именно КНФ: в ДНФ выполнимость читается по первому истинному терму, и задача исчезает.

Решатели живут в КНФ, где дизъюнкты взаимодействуют через общие переменные.

Цена дистрибутивности

Теорема 2 (Взрыв размера)

КНФ формулы $(a_1 \wedge b_1) \vee \dots \vee (a_k \wedge b_k)$ содержит ровно 2^k дизъюнктов, каждый длины k .

Доказательство

Докажем индукцией по числу пар k .

База. При $k = 1$ формула $a_1 \wedge b_1$ уже КНФ из двух единичных дизъюнктов, и их ровно 2^1 .

Шаг. Дистрибуция $(C_1 \wedge \dots \wedge C_s) \vee X = (C_1 \vee X) \wedge \dots \wedge (C_s \vee X)$ превращает КНФ F_{k-1} с 2^{k-1} дизъюнктами длины $k - 1$ в пару КНФ $(F_{k-1} \vee a_k)$ и $(F_{k-1} \vee b_k)$.

Каждый из 2^{k-1} старых дизъюнктов даёт двух наследников — с a_k и с b_k , итого $2 \cdot 2^{k-1} = 2^k$ дизъюнктов длины k .

Взрыв на числах

Пример

При $k = 3$ восемь дизъюнктов: $(a_1 \vee a_2 \vee a_3), (a_1 \vee a_2 \vee b_3), \dots, (b_1 \vee b_2 \vee b_3)$.

Каждый дизъюнкт выбирает по одному литералу из каждой пары, и все 2^3 выбора различны.

При $k = 300$ дизъюнктов $2^{300} \approx 2 \times 10^{90}$ — на двадцать порядков больше, чем атомов во вселенной.

«Любую формулу можно привести к КНФ без роста размера» — неверно.

Эквивалентное преобразование способно раздуть формулу экспоненциально, и формула из теоремы — не патология, а норма.

Спасение другое: решателю не нужна равносильность, достаточно равновыполнимости — так работает преобразование Цейтина.

Перебор экспоненциален

Простейший алгоритм — перебрать все наборы:

n переменных $\rightarrow 2^n$ наборов.

Пример

- $n = 20$: около миллиона наборов — мгновенно.
- $n = 40$: около триллиона — часы счёта.
- $n = 300$: наборов больше, чем атомов во вселенной.

Перебор можно ужать отсечениями, но экспонента в худшем случае остаётся.

Вопрос «можно ли быстрее?» — центральный вопрос всей теории сложности.

2-SAT и граф импликаций

Ограничим дизъюнкты длиной два: задача k -SAT отличается от SAT ровно тем, что каждый дизъюнкт содержит ровно k литералов.

При $k = 2$ у формулы появляется структура, которую видит теория графов.

Определение 10 (Граф импликаций)

Вершины — все литералы $x_1, \overline{x_1}, \dots, x_n, \overline{x_n}$.

Каждый дизъюнкт $(l_1 \vee l_2)$ даёт два ребра: $\overline{l_1} \rightarrow l_2$ и $\overline{l_2} \rightarrow l_1$.

Дизъюнкт ложен только когда ложны оба литерала, поэтому из ложности l_1 следует истинность l_2 — это первое ребро.

Второе ребро — контрапозиция, тот же дизъюнкт, прочитанный с другого конца, и терять его нельзя: на урезанном графе компоненты сильной связности (далее — КСС) меняются, а вместе с ними и ответ задачи.

Критерий 2-SAT

Теорема 3

Формула 2-SAT невыполнима тогда и только тогда, когда пара $x, \bar{x}$ лежит в одной КСС графа импликаций.

Доказательство

Докажем эквивалентность разбором обеих сторон. Пусть x и $\bar{x}$ лежат в одной КСС: есть путь из x в $\bar{x}$ и обратный, а всякая модель удовлетворяет каждому ребру графа. При $x = 1$ путь из x даёт $\bar{x} = 1$, при $x = 0$ обратный путь даёт $x = 1$: модели нет.

Обратно: если пары разнесены, сожмём граф в конденсацию и объявим истинными литералы, чьи компоненты идут позже компонент их отрицаний. Вдоль пути компоненты не убывают, поэтому истинный литерал тянет истинного соседа: все дизъюнкты выполнены.

Граф импликаций: полный проход

Алгоритм: построить граф, найти КСС Косарайю или Тарьяном, назначить истину по конденсации — линейно, $O(n + m)$.

Прогоним его на ψ из таблицы выше.

1. Дизъюнкт $(x \vee y)$ даёт рёбра $\bar{x} \rightarrow y$ и $\bar{y} \rightarrow x$, дизъюнкт $(\bar{x} \vee y)$ — рёбра $x \rightarrow y$ и $\bar{y} \rightarrow \bar{x}$.
2. Цикла через x и $\bar{x}$ нет: все КСС одноэлементны, формула выполнима.
3. В конденсации $\bar{y}$ — источник, y — сток, поэтому истину назначаем от стоков: $y = 1$.
4. x и $\bar{x}$ не сравнимы — x свободен, и модели $(0, 1)$, $(1, 1)$ совпадают с таблицей.

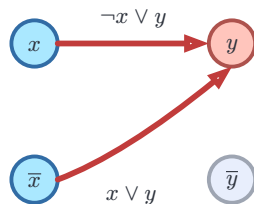


Рис. 1. Граф импликаций ψ : передние рёбра, контрапозиции опущены.

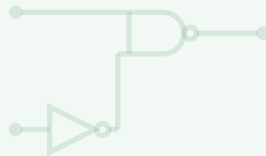
2-SAT против 3-SAT

Третий литерал рушит конструкцию: из ложности l_1 дизъюнкт $(l_1 \vee l_2 \vee l_3)$ требует лишь $l_2 \vee l_3$ — дизъюнкт, а не литерал.

Импликация между дизъюнктами не даёт ни графа на литералах, ни компонент сильной связности — вся техника 2-SAT исчезает.

Поэтому граница проходит ровно здесь: 2-SAT разрешима за линейное время, а 3-SAT NP-полна.

Разбивка длинного дизъюнкта свежей переменной тоже не спасает: части $(l_1 \vee l_2 \vee y)$ и $(\bar{y} \vee l_3 \vee l_4)$ снова содержат трёхлитеральные дизъюнкты, и задача не становится 2-SAT.



Решатели

Давайте вычислим!

— Готфрид Лейбниц

От перебора к DPLL

Алгоритм DPLL (Дэвис, Патнэм, Логеман, Лавленд, 1962) — поиск с возвратом плюс два приёма, которые срезают ветви до перебора. Общая идея: перед каждым ветвлением формула выжимает всё из уже прочитанного. Значения, которые не выбираются, а принуждаются, расставляются сами.

- **Ветвление:** выбрать неразобранную переменную и развести случаи $x = 0$ и $x = 1$.
- **Распространение единичного дизъюнкта:** если все литералы дизъюнкта, кроме одного, уже ложны, оставшийся принуждён — иначе дизъюнкт ложен целиком. Дальше значения разлетаются каскадом без всякого выбора.
- **Чистые литералы:** переменная, встречающаяся только с одной полярностью, безопасно получает выполняющее значение — такой выбор не ломает ни один дизъюнкт.

Оба приёма сохраняют выполнимость: формула после них выполнима тогда и только тогда, когда была выполнима исходная.

Единичное распространение каскадом

Пример (Каскад без ветвления)

Разберём формулу из четырёх дизъюнктов: $F = (x) \wedge (\bar{x} \vee y) \wedge (\bar{y} \vee z) \wedge (\bar{z})$.

Единичный дизъюнкт (x) принуждает $x = 1$. В дизъюнкте $(\bar{x} \vee y)$ литерал $\bar{x}$ стал ложным, и дизъюнкт сжался до единичного: $y = 1$. Тот же механизм даёт $z = 1$ из $(\bar{y} \vee z)$. Теперь единичный $(\bar{z})$ ложен целиком — конфликт. Формула невыполнима, и ни одного ветвления не понадобилось: распространение само довело перебор до противоречия.

На практике распространение — самая дорогая часть решателя: по измерениям авторов Chaff это свыше 90% времени.

Чистые литералы и цена DPLL

Пример (Чистый литерал)

В формуле $(x \vee y) \wedge (\bar{x} \vee z)$ переменные y и z встречаются только в положительной полярности. Присваивание $y = 1, z = 1$ выполняет оба дизъюнкта при любом значении x . Перебор по x отменяется — формула выполнима.

Отсечения ужимают перебор в разы, но в худшем случае DPLL экспоненциален.

Насколько тяжёл худший случай — измеряет теорема Кука — Левина.

Теорема Кука — Левина

Теорема 4 (Кук — Левин)

SAT NP-полна: выполнимость проверяется за полиномиальное время по сертификату, и всякая задача класса NP полиномиально сводится к SAT.

Доказательство (набросок)

Сертификат — выполняющий набор: подстановка значений в КНФ стоит $O(|\varphi|)$.

Трудная половина — сводимость: недетерминированная машина работает $p(n)$ шагов, и её вычисление разворачивается в таблицу $p(n) \times p(n)$.

Переменные кодируют символ ячейки, состояние и позицию головки на каждом шаге, а переход локален: окно 2×3 таблицы даёт константное число дизъюнктов.

Дизъюнктов $O(p(n)^2)$, построение полиномиально, и формула выполнима тогда и только тогда, когда машина принимает вход.

Цена теоремы

Следствие 1 (Цена вопроса)

Полиномиальный алгоритм для SAT дал бы полиномиальные алгоритмы для всех задач из NP: свести — и запустить.

Поэтому вопрос $SAT \in P$ — это в точности вопрос $P = NP$.

Вопрос $P = NP$ — одна из семи задач тысячелетия: институт Клэя назначил за любой ответ миллион долларов.

NP-полнота на практике

NP-полнота — не вердикт «не решать», а карта: через SAT проходит любая задача с быстрой проверкой ответа.

Один хороший решатель — общий движок для расписаний, верификации схем, планирования и головоломок.

Карп в 1972 году свёл SAT к двум десяткам задач — от раскраски графа до покрытия множества, положив начало длинной цепочке NP-полных задач.

Сегодня счёт идёт на тысячи: NP-полнота новой задачи доказывается цепочкой сведений, начинающейся с SAT.

Полнота против конкретного экземпляра

NP-полнота описывает худший случай, а не конкретный экземпляр.

Отсутствие полиномиального алгоритма не мешает решателю выдать модель за миллисекунды — для индустриальных формул это норма.

Пример

У sudoku 9×9 наивно $9^{81} \approx 2 \times 10^{77}$ заполнений, и решатель не трогает и ничтожной доли.

Ограничения-дизъюнкты отсекают целые ветви перебора задолго до экспоненты.

Худший случай против практики

Пример (Принцип Дирихле как КНФ)

Голубей $n + 1$, ячеек n : переменная $p_{i,j}$ означает «голубь i сидит в ячейке j ».

Дизъюнкты: каждый голубь где-то сидит — $n + 1$ штук, никакие два не делят ячейку — $\binom{n+1}{2} \cdot n$ штук, всего полиномиально много.

Невыполнимость очевидна человеку, но всякий резолюционный вывод пустого дизъюнкта содержит $2^{\Omega(n)}$ дизъюнктов.

Дизъюнктов в формуле полиномиально много — экспоненциальна только длина доказательства.

Размер записи и краткость вывода — разные ресурсы, и трудность живёт во втором.

Структура против случайности

Случайный 3-SAT имеет фазовый переход по отношению дизъюнктов к переменным около $\frac{m}{n} \approx 4.267$: ниже порога почти наверняка SAT, выше — UNSAT.

На самом пороге экземпляры труднее всего — именно там перебор ведёт себя как настоящая экспонента.

Индустриальные формулы структурированы: сообщества переменных, малая ширина дерева зависимостей, короткие наборы «ключевых» переменных.

CDCL находит эту структуру выученными дизъюнктами — и решает формулы с миллионами переменных за секунды.

Кодирование в SAT: стратегия

1. Выбрать булевы переменные, описывающие решение.
2. Записать ограничения задачи дизъюнктами.
3. Запустить SAT-решатель.
4. Модель декодировать в решение, UNSAT — решения нет.

Ответ UNSAT — часто самое ценное: он доказывает несуществование решения, чего не даст ни один тест.

Хорошая кодировка наследует структуру задачи.

Плохая раздувает формулу и останавливает решатель.

Преобразование Цейтина

Определение 11 (Преобразование Цейтина)

Преобразование Цейтина переводит формулу или схему в равновыполнимую КНФ: каждый выход вентиля получает свежую переменную, а связь с входами фиксируется константным числом дизъюнктов.

Равновыполнимость: модели КНФ на исходных переменных — в точности модели исходной формулы.

Стандартные вентили — по таблице: дизъюнкты запрещают ровно неверные комбинации.

Вентиль	Функция	Дизъюнкты
NOT	$y = \bar{a}$	$(y \vee a), (\bar{y} \vee \bar{a})$
AND	$y = a \wedge b$	$(\bar{y} \vee a), (\bar{y} \vee b), (y \vee \bar{a} \vee \bar{b})$
OR	$y = a \vee b$	$(y \vee \bar{a}), (y \vee \bar{b}), (\bar{y} \vee a \vee b)$

Цейтин: пример

Пример (Один клапан целиком)

Схема $y = \bar{a} \wedge b$ — строка AND с инвертированным входом a , плюс головной единичный дизъюнкт:

$$(\bar{y} \vee \bar{a}) \wedge (\bar{y} \vee b) \wedge (y \vee a \vee \bar{b}) \wedge (y).$$

Единичный дизъюнкт даёт $y = 1$, распространение — $a = 0$ и $b = 1$.

Модель совпала с единственным набором, на котором клапан выдаёт единицу.

Дизъюнкты Цейтина разрешают ровно те комбинации (a, b, y) , при которых $y = \bar{a} \wedge b$, — связь двусторонняя.

Поэтому модель схемы и модель КНФ переводятся друг в друга назначением вспомогательных переменных.

Цейтин: линейность

Теорема 5

Преобразование Цейтина даёт КНФ размера $O(|\varphi|)$ — не более одной свежей переменной и трёх дизъюнктов на каждый вентиль.

Доказательство

Докажем подсчётом по структуре формулы.

Листья формулы — переменные, и они уже литералы. Каждая внутренняя вершина — один вентиль.

На вентиль заводится одна переменная и не более трёх дизъюнктов константной длины, на головную вершину — единичный дизъюнкт.

Переменных не больше $n + t$, дизъюнктов не больше $3t + 1$, где t — число вентиляй: размер линеен.

Равновыполнимость, не равносильность

КНФ Цейтина не равносильна исходной формуле: вспомогательные переменные делают формулы разными объектами.

Она равновыполнима, поэтому подставлять её, например, под отрицание нельзя: при неверных значениях вспомогательных переменных КНФ ложна даже там, где исходная формула истинна.

Равносильность нужна при встраивании формулы в большее выражение, а решателю SAT достаточно выполнимости — на этом Цейтин законно экономит.

Что мы поняли: булевы функции и SAT

Булевых функций от n переменных ровно 2^{2^n} , и таблица — единственный способ задать произвольную. Минтермы и макстермы собирают совершенные формы — единственные, но избыточные.

SAT — вопрос о существовании модели: наборов 2^n , дизъюнкт длины k запрещает 2^{n-k} из них. Эквивалентное приведение к КНФ взрывается как 2^k , и его обходит Цейтин — ценой отказа от равносильности.

2-SAT решается за линейное время: дизъюнкт — пара импликаций, невыполнимость — пара $x, \bar{x}$ в одной КСС. Третий литерал рушит структуру, и 3-SAT уже NP-полна.

DPLL срезает перебор распространением единичных дизъюнктов и чистыми литералами, но в худшем случае экспоненциален. Кук — Левин ставит SAT во главу NP, а индустрия решает его на миллионах переменных — структур худшего случая в живых задачах нет.

Вопросы для самопроверки

Вопросы для самопроверки

1. Постройте СДНФ и СКНФ для $f(x, y) = x \rightarrow y$. Совпадает ли минимальная ДНФ с одной из совершенных форм?
2. Закодируйте в КНФ требование «ровно одна из переменных x, y, z истинна».
3. Почему КНФ Цейтина лишь равновыполнима с исходной формулой? Что сломается при подстановке её результата под отрицание?
4. Почему дизъюнкт $(l_1 \vee l_2)$ обязан породить в графе импликаций оба ребра? Что изменится, если оставить одно?
5. Как по компонентам сильной связности графа импликаций построить выполняющий набор?
6. Назовите два приёма DPLL до ветвления и объясните, почему каждый безопасен.

Что почитать

- m10 — «Булевы функции», «ДНФ и КНФ», «СДНФ, АНФ и смена базиса».
- m14 — «Задача SAT», «Алгоритм для 2-SAT», «От DPLL к CDCL», «Кодирование задач в SAT».